

—— // ZSHRS // ——

# znative

first edition

---

## THE MANAGER

the published ABI · the content-addressed store · native code as a package

2026-07-18

MenkeTechnologies

—— MENKETECHNOLOGIES ——

# Contents

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>Before You Begin</b>                                    | <b>2</b>  |
| <b>Chapter 1 — The Gap</b>                                 | <b>4</b>  |
| <b>Chapter 2 — The Published ABI</b>                       | <b>5</b>  |
| <b>Chapter 3 — What znative Is</b>                         | <b>7</b>  |
| <b>Chapter 4 — The Command Surface</b>                     | <b>9</b>  |
| <b>Chapter 5 — Sources and Resolution</b>                  | <b>10</b> |
| <b>Chapter 6 — The Content-Addressed Store</b>             | <b>11</b> |
| <b>Chapter 7 — Kind Detection</b>                          | <b>13</b> |
| <b>Chapter 8 — The Manifest: <code>znative.toml</code></b> | <b>14</b> |
| <b>Chapter 9 — The <code>.zshrc</code> Idiom</b>           | <b>15</b> |
| <b>Chapter 10 — The Worked Ports</b>                       | <b>16</b> |
| <b>Chapter 11 — The Landscape</b>                          | <b>17</b> |
| <b>Chapter 12 — Lineage: the Stryke Dependency Manager</b> | <b>18</b> |
| <b>Chapter 13 — What znative Is Not</b>                    | <b>19</b> |
| <b>Appendix A — Command Reference</b>                      | <b>20</b> |
| <b>Appendix B — Source-Spec Grammar</b>                    | <b>21</b> |
| <b>Appendix C — Store Layout</b>                           | <b>22</b> |
| <b>Appendix D — <code>znative.toml</code> Schema</b>       | <b>23</b> |
| <b>Appendix E — The Doctrine</b>                           | <b>24</b> |

# Before You Begin

This is the book about `znative` — the one you read in a chair, front to back, to understand not only *what* the tool does but *why* it is shaped the way it is. It is not the README, which enumerates every subcommand, path, and flag and which you reach for when you need one fact fast. This book is the other thing: the continuous account of a package manager whose unit of installation is compiled native code rather than shell text, and of the one guarantee — a published, versioned ABI — that makes installing that native code safe.

`znative` is the zshrs package manager, and the first shell package manager whose unit of installation can be native machine code rather than shell text. It is not a fork of an existing plugin manager and it is not a wrapper around `git clone`. It is a small, global-only manager built into `zshrs` — the first compiled Unix shell — and it installs two kinds of plugin from one command surface: the script plugins every zsh user already knows (oh-my-zsh, zinit bundles, loose `.zsh` files) and compiled Rust `cdylib` plugins that load at native speed through a stable ABI. Every prior shell plugin manager does exactly one thing — clone a git repository and source its `.zsh` files. `znative` does that too, and it also does the thing none of them can: install a compiled plugin.

The reason no shell package manager before `znative` installed native code is not an oversight; it is a consequence. Until zshrs published its native-plugin interface as an independently-versioned ABI, there was nothing safe to install — a native shell module compiled against a shell's private internal headers is welded to one exact build and can crash a mismatched one. This book spends its early chapters on that problem, because the manager only makes sense once the ABI underneath it does, and the two ship together as one invention in two halves: a published ABI makes a native plugin *buildable* by a stranger; a package manager makes it *installable* by one.

The whole workflow collapses to a single self-installing line in a `.zshrc`:

```
znative load MenkeTechnologies/zshrs-forgit
```

On the first shell start that line resolves the source, installs the plugin into a content-addressed store, and loads it. On every start after, it loads from the store with zero network. There is no separate install step, no second tool, and no second mental model layered on top of the plugin workflow zsh users already have.

This is fast-moving software. `znative` is its own crate — the zshrs package manager — so the edition string on the cover tracks the `znative` crate version. Where a behaviour depends on the compiled-plugin path — the ABI, the `cdylib` build, the native registry — this book marks it, because the line between the script path and the native path *is* the design.

## How to read this book

The book runs bottom-up. The first chapters are the pitch and the justification — the gap in every existing shell, what `znative` is, and why it required a stable ABI to exist at all. Then it descends the stack: the published ABI and the `zmodload -R` host it loads plugins through; the command surface; source resolution; the content-addressed store; kind detection; the optional `znative.toml` manifest; and the `.zshrc` idiom that ties it together. After the mechanism come the worked plugin ports that prove the path end to end, the landscape of prior plugin managers `znative` is measured against, and the stryke dependency manager it was ported from. The reference appendices at the back cover the full command table, the source-spec grammar, the store layout map, the manifest schema, and the shell doctrine the tool obeys.

Every claim in the chapters is grounded in the zshrs implementation and in the invention ledger that first

recorded [znative](#) as a “world’s first.” Where the tool deliberately *omits* something other package managers have — a lockfile, a per-project manifest, a global registry — the book says so and says why, because the omissions are as considered as the features.

# Chapter 1 – The Gap

Every shell has an extension story, and until now they came in exactly two shapes. Neither of them installs native code, and the reason why is the whole starting point for this book.

The first shape is the **script plugin**. This is the oh-my-zsh / zinit / antigen / sheldon / zplug world, and it is where essentially all real shell customization lives. A script plugin is a git repository of `.zsh` files. A plugin manager clones the repository and `sources` those files into the running shell, adding functions, aliases, completions, and key bindings. It is text the shell reads and re-parses, and every one of these managers, underneath its own configuration DSL, does the same single mechanical act: clone a repo, `source` its files. They differ in ergonomics — turbo/deferred loading, lockfiles, declarative blocks — but not in kind. The unit of installation is shell text, interpreted on load and, for anything that runs per prompt or per keystroke, re-interpreted forever after.

The second shape is the **native module**. zsh has `zmodload`, which loads a compiled C module (`zsh/zle`, `zsh/complist`, `zsh/mathfunc`, and the rest); bash has `enable -f`, which loads a compiled “loadable builtin.” These run as machine code — no interpreter cost — and that is their entire appeal. But they compile against the shell’s *internal, private* headers, with no stable ABI and no version gate. A zsh C module is built against `Src/` — the shell’s own build tree — so it is welded to one exact shell build. Load it into a shell compiled from even a slightly different source and it can crash the process, because the struct layouts, function signatures, and global symbols it was compiled against no longer match the host it was loaded into.

That single fact explains something you can verify by looking: **neither ecosystem has a meaningful third-party native-module scene**. Almost every native module that exists for zsh is one the shell itself ships in its own source tree, because tracking a moving internal API from *outside* the source tree is not viable. A stranger cannot reliably build a zsh C module against your zsh and hand it to you; the moment either of you rebuilds the shell, the module may stop loading or start corrupting memory. The native path was fast but unshippable by third parties. The script path was shippable by anyone but interpreted forever.

So a shell package manager, historically, had only one thing it could safely install: shell text. That is not a design choice any of those managers made — it is the only option the substrate gave them. There was no third shape to install, because there was no safe way to *distribute* native shell code at all. The gap this book is about is the gap where that third shape should be: a compiled plugin a stranger can build and you can install, running at native speed, that cannot crash your shell because it was built somewhere else.

Closing that gap takes two pieces that do not exist independently. You need a way to *build* a native plugin against a shell without cloning the shell’s source and without welding to one build — a stable, published interface. And you need a way to *install* the result — a package manager that understands a compiled artifact as a unit, not just a bag of `.zsh` files. `znative` is the second piece. The next chapter is the first.

## Chapter 2 – The Published ABI

`znative` installs native plugins, and it can only do that because there is something safe to install. That something is zshrs's native-plugin ABI, published as an independently-versioned package, and it is the invention the package manager sits on top of. This chapter is the foundation; the manager is the rest of the book.

Recall the trade zshrs makes with zsh's own module system. zsh loads C modules at runtime with `zmodload`; zshrs instead statically links its fourteen-plus built-in modules and decides the whole module surface at link time, so a `zmodload` of a name the binary knows can never fail for reasons outside the binary itself. That trade gives up the ability to load a foreign `.so` — and with it every failure mode that came bundled with that ability: version skew, missing symbols discovered only at load time, dependence on the host's dynamic linker behaving identically across macOS and both Linux architectures, and the security surface of loading arbitrary shared objects out of `$module_path`. For the built-in modules, giving up dynamic loading is the conservative, correct default.

But there is a version of dynamic loading that carries none of those failure modes, and zshrs adds it as a **separate, additive path**. Native plugins are written in a compiled language — Rust — and loaded at runtime with `zmodload -R <path>`, with no recompile of the shell. The `-R` flavour sits beside the ordinary `zmodload`, not in place of it: the built-in modules stay statically linked and compile-time-proven; the plugin path is the one place a foreign compiled artifact enters the process, and it enters through a gate the built-in modules never had.

The gate is the invention: **a stable, versioned, independently-published ABI**. zshrs exposes its plugin interface not as its own build-tree internals but as the `znative` crate — the same crate this book is about, published from the zshrs repo to crates.io. `znative` is one crate wearing two hats: the package manager the user drives, and the plugin ABI a plugin author builds against. A plugin author runs `cargo add znative`, writes a handler against that crate's public types, and ships a `cdylib`. The author never clones zshrs. Everything that crosses the boundary between host and plugin is `#[repr(C)]` — a stable, compiler-independent memory layout, not Rust's unspecified default layout — and the host checks the plugin's `ABI_VERSION` against its own before it trusts a single pointer the plugin hands back. A version mismatch is *refused at load*, not left to corrupt the process later. That one check is the whole difference between “load a foreign `.so`” as the liability the built-in module system rejects and “load a foreign `.so`” as a safe, first-class extension path.

Mechanically the load is small. `zmodload -R libfoo.dylib dlopens` the library (through the `libloading` crate), resolves a single exported symbol — `znative_plugin_init` — and calls it with a host handle. The plugin's `init` registers its builtins into an in-process registry and, from then on, reaches the shell only through a curated host API — `print`, `eval`, `getvar`, `setvar` — rather than reaching into shell internals the way a C module does. The plugin never touches a shell global directly; it asks the host to do things on its behalf through four functions, all of them `#[repr(C)]` across the boundary. This is the second half of why the ABI is stable: not only is the layout fixed and version-checked, the *surface* the plugin can touch is a deliberately small, curated set of calls, not the shell's entire internal state.

Command resolution then slots the plugin registry into the shell's dispatch at a specific, principled point. When zshrs resolves a command, the plugin registry is consulted in `execute_external_bg ahead of PATH lookup` — the same place zsh resolves `zmodload -ab` autoloaded builtins. The effect is exact: a plugin command shadows an external program of the same name on `$PATH`, but it never shadows a real builtin or a shell function. A plugin can give you a faster `git` front-end; it cannot silently replace `cd` or a function you defined. Unloading is the mirror image: `zmodload -uR` purges the plugin's entries from the registry *before* the library's `dlclose`, so no dangling function pointer into a just-unmapped library can survive to be called.

The distinction that matters here is not “loads native code” — bash and zsh have loaded native code for decades. It is that **zshrs is the first shell to publish that interface as a stable, versioned package a stranger can build against without ever cloning the shell.** That is what makes native shell plugins *distributable*. And the moment a thing is distributable, it wants a distributor. That distributor is **znative**.

# Chapter 3 – What znative Is

**znative** is the zshrs package manager. In one sentence: it resolves a plugin source, installs it into a content-addressed store, records it in a global index, and loads it — for script plugins and compiled native plugins alike, from one small command surface. It is the first shell package manager whose unit of installation can be native code rather than shell text, and that “can be” is the whole point: it installs the entire existing zsh script-plugin ecosystem unchanged *and* installs the new thing the published ABI made possible.

Hold the two halves of the invention side by side, because **znative** only makes sense as the second one. A published ABI (Chapter 2) makes a native plugin **buildable** by a stranger. A package manager makes it **installable** by one. Without the ABI, **znative**’s native path would have nothing safe to install and would collapse back into “clone a repo and source it” — which is all any prior manager ever did. Without **znative**, the ABI would let a stranger build a compiled plugin that every user then had to place, build, and **zmodload -R** by hand. The two ship as one invention because each is inert without the other.

What **znative** deliberately is *not* is as defining as what it is:

- **It is not a fork of oh-my-zsh, zinit, antigen, sheldon, or zplug.** It shares their script-plugin behaviour — clone a repo, add its files to `$fpath`, source them — because that behaviour is correct and universal, but it is a fresh implementation with a different core: a content-addressed store and a native install path those managers have no concept of.
- **It is not global-and-project like a language package manager.** There is no per-project manifest and no lockfile. **znative** is global only, because a shell’s plugins are part of the *user’s environment*, not a *project’s* dependency set. Chapter 11 makes the case for that omission in full.
- **It is not a new configuration DSL.** The primary interface is one line in a `.zshrc` — **znative load owner/repo** — not a declarative block with its own syntax to learn. The tool’s surface is small on purpose.

The creed for the whole thing is the self-installing line from the front matter: **znative load MenkeTechnologies/zshrs-forgit**. That single line, in a `.zshrc`, is a complete plugin lifecycle. First start: resolve, install, load. Every start after: load from the store, no network. It is the script-plugin workflow every zsh user already knows — extended, without a second tool or a second mental model, to compiled native plugins. The rest of this book is what happens underneath that line.

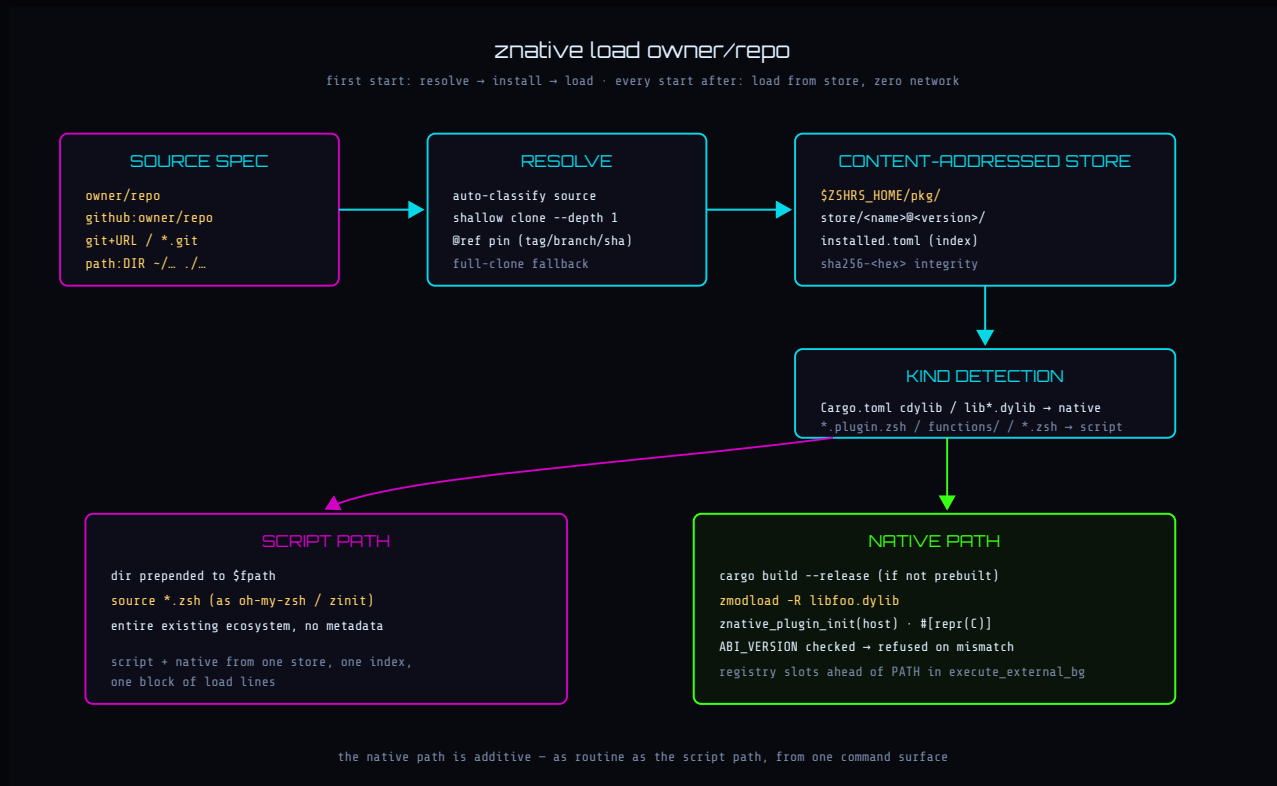


Figure 1: **znative** installs from a source spec into a content-addressed store, then loads a script plugin onto **\$fpath** or a native **cdylib** through the **zmodload -R** ABI host. The first start installs; every start after loads from the store with zero network.

# Chapter 4 – The Command Surface

The command surface is deliberately small — six verbs, each with a single job, all obeying the same terse error contract as the rest of the shell. This chapter is the tour; the appendix at the back is the table.

`znative load` is the verb a `.zshrc` calls, and it is the one that makes the whole thing collapse to a single line. Given a name, or a source already present in the store, it loads that plugin with zero network. Given a source *not* yet in the store, it installs it first and then loads it. That conditional is the entire reason there is “no separate install step” for the common case: the first shell start after you add a `load` line does the install as a side effect of the load; every start after finds the plugin already in the store and just loads it. A bare `znative load` with no argument is a special, useful case — it loads everything already present in the store, for the user who prefers to install interactively with `znative add` and then keep a single unqualified `load` line in the startup file.

`znative add` (aliases `install`, `i`) is the interactive install. It resolves the source, installs it into the store, records it in the global index, and loads it — all without needing a line in `.zshrc`. This is what you run at a prompt when you want to try a plugin now; if you then want it every session, you add a `load` line for it, and because it is already in the store that line will never hit the network again.

`znative remove` is the clean uninstall. For a native plugin it unloads the library first (the `zmodload -uR` purge-then-`dlclose` from Chapter 2), then it deletes the plugin’s copy from the store and drops its row from the index. The order matters: unload before delete, so nothing is executing out of a directory that is about to disappear.

`znative list` prints one line per installed plugin — name, version, kind, and source. It is the fast “what do I have” read, and it comes straight off the global index rather than by scanning the filesystem, so it reflects exactly what `znative` believes is installed.

`znative info` is the deep read for one plugin. It shows the full record: the store path, the integrity hash, and the specific lib (for a native plugin) or the source files (for a script plugin) that the plugin resolves to. When you want to know precisely what a `load` line will pull in, this is the verb.

`znative update` re-resolves and reinstalls from the *recorded source* — the source spec `znative` saved when it first installed the plugin — for one named plugin or, with no argument, for all of them. Because the recorded source can carry a pin (Chapter 5), `update` respects it: updating a plugin pinned to `@v1.2.0` re-fetches that tag, not a moving branch.

The error contract is uniform and terse. Anything that fails prints `znative: <reason>` on `stderr` and returns non-zero. There is no friendly explanation, no “did you mean,” no “run `-help` for more.” It is the same contract the rest of `zshrs` uses — state the error, exit — because the tool is built for a user who has typed these commands a million times and wants the failure, not a lecture about it.

# Chapter 5 – Sources and Resolution

A **znative** source is a single string, and its form is **auto-classified** rather than flagged. You do not tell **znative** “this is a GitHub repo” or “this is a local path” with an option; it reads the shape of the string and decides. That keeps the common case — a GitHub **owner/repo** — down to the shortest thing that could possibly identify a plugin.

The classifications, in the order a reader would recognise them:

- **owner/repo** — a bare two-part slug with no scheme is a GitHub repository. **znative load ohmyzsh/ohmyzsh** clones from GitHub. This is the form nearly every **.zshrc** line uses.
- **github:owner/repo** — the same thing said explicitly, for when you want the host named in the source rather than inferred.
- **a git+URL, or any URL ending in .git** — clone that URL directly. This is the escape hatch to any git host: GitLab, a self-hosted forge, an internal mirror. **git+https://git.example.com/team/plugin.git** clones from there.
- **path:DIR, or an absolute / ./ / ~-rooted path** — install a local directory with **no network at all**. This is how you develop a plugin: point **znative** at the working tree, and it installs from disk. It is also how a fully offline environment installs plugins it already has on the filesystem.

Any remote form may carry an **@ref suffix** to pin what gets fetched: **owner/repo@v1.2.0**, **github:owner/repo@some-branch**, **git+https://host/x.git@main**, or a suffix naming a specific commit. The pin is a tag, a branch, or a commit — whatever git can check out — and it is recorded with the plugin so that a later **znative update** re-fetches exactly that ref rather than drifting to the head of a default branch.

The clone itself is optimised for the common case and correct in the uncommon one. **znative** clones **shallow** — **--depth 1 --branch REF** — because a plugin install needs the tree, not the history. It only falls back to a full clone when a shallow branch clone cannot reach a requested commit: pinning to an arbitrary older commit sometimes requires history a **--depth 1** fetch will not have, and in that case **znative** does the full clone rather than fail. The default path is fast (one shallow fetch); the fallback exists so that pinning to any commit always works.

Whatever the source, once the tree is on disk the next question is *what kind of plugin is this* — and that is Chapter 7. But first, where the tree goes.

# Chapter 6 – The Content-Addressed Store

Everything `znative` installs lives under **one store**, at `$ZSHRS_HOME/pkg/` (default `~/.zshrs/pkg/`). There is exactly one store per user; there is no per-project `.znative/` directory anywhere, because `znative` is global only. This chapter is the map of that directory and the index that governs it.

The store's layout:

- `store/<name>@<version>/` — the installed plugin itself, keyed by name and version. The tree here is the plugin *stripped for use*: `.git/` and `target/` are removed, so the store holds the source and any built library but not the version-control history or the Rust build's intermediate artifacts. That keeps the store small and makes each `store/<name>@<version>/` a clean, self-contained copy of exactly what gets loaded.
- `installed.toml` — the **global index and the source of truth**. Every installed plugin has a record here: its name, version, kind, recorded source spec, store path, and integrity hash. `znative list` and `znative info` read this file; `znative add` writes to it; `znative remove` drops a row from it. When `znative` and the filesystem disagree, this file is what `znative` believes.
- `git/`, `cache/`, `bin/` — internal scratch. Clone working directories, fetch caches, and helper binaries live here. These are `znative`'s workspace, not part of any plugin's installed footprint.

Each install is **content-addressed**: the record carries a SHA-256 hash written as `sha256-<hex>`, so a plugin's index row is its own integrity check. The store is not a bag of directories you have to trust by their names — every entry knows its own hash, and that hash is what `znative info` surfaces when you ask what a plugin actually is. Content addressing is why the store can be shared, inspected, and trusted: the identity of an installed plugin is the hash of its content, not just the label on its folder.

And then the deliberate omissions, which are the design as much as the structure: **there is no per-project manifest and no lockfile**. A language package manager (cargo, npm, bundler) has both, because a project's dependency set must be reproducible across machines and pinned in version control. A shell's plugins are not that. They are part of *your environment* — the set of tools you, the human, have chosen to have at your prompt — not part of a project you hand to a collaborator. So `znative` keeps one global store and one global index, and it does not pretend a shell's plugin set is a project's lockfile. Chapter 11 returns to why “global only” is the right model rather than a missing feature.



Figure 2: The store at `$ZSHRS_HOME/pkg/`: `store/<name>@<version>/` holds each stripped plugin tree, `installed.toml` is the global index and source of truth (name · version · kind · source · `sha256-` hash), and `git/`, `cache/`, `bin/` are internal scratch.

# Chapter 7 — Kind Detection

`znative` installs two kinds of plugin — **native** (a compiled Rust `cdylib`) and **script** (zsh text on `$fpath`) — and when the repository does not declare which it is, `znative` detects the kind from the shape of the tree. Detection is what lets the entire existing zsh ecosystem install with no metadata at all, and it is what lets a Rust plugin install with nothing more than a `Cargo.toml`.

A plugin is detected as **native** when either of two signals is present:

- a prebuilt `lib*.{dylib,so}` at the repository root — a library already compiled, ready to load; or
- a `Cargo.toml` whose `[lib] crate-type` includes `cdylib` — a Rust plugin meant to be built into a loadable library.

A native plugin loads through the `zmodload -R` ABI host of Chapter 2. When a prebuilt library is present, `znative` loads it directly. When only the `Cargo.toml` is present, `znative` builds it first — `cargo build --release` — and then loads the resulting `cdylib`. Either way, what ends up loaded is machine code crossing the `#[repr(C)]`, version-checked boundary, not interpreted shell text.

A plugin is detected as **script** when the native signals are absent and any of these are present:

- a `*.plugin.zsh` file (the oh-my-zsh convention);
- a `functions/` directory (the completion/autoload convention); or
- loose `*.zsh` files at the root.

A script plugin is handled exactly as oh-my-zsh or zinit would handle it: its directory is added to `$fpath` and its files are `sourced` as-is. This is the point of the whole detection scheme — **the entire existing script-plugin ecosystem installs with no metadata whatsoever**. You do not write a manifest to install `zsh-autosuggestions`; `znative` sees the `.zsh` files, classifies it as a script plugin, and sources it, because that repository already looks exactly like what a script plugin is.

Detection covers the common cases. When a repository wants to be *explicit* — to override detection, or to declare details detection cannot infer — it ships a manifest. That is the next chapter.

# Chapter 8 — The Manifest: `znative.toml`

Auto-detection (Chapter 7) is enough for the common cases, but a repository that wants to be precise ships a `znative.toml` at its root. The manifest **overrides** detection; where it is present, `znative` reads it rather than guessing. Where it is absent, detection covers the rest. Nothing about the manifest is required — it is the explicit path for plugins that need it, not a barrier every plugin must clear.

The manifest has a `[plugin]` table and one of two kind tables.

`[plugin]` carries the metadata that identifies the plugin regardless of kind:

- **name** — the plugin's name, used as the `<name>` in the store path and the index.
- **version** — the version, used as the `<version>` in `store/<name>@<version>/`.
- **description** — a human-readable one-liner.

Then exactly one of the kind tables:

`[native]` declares a compiled plugin. Its central field names the **lib stem** — the base name of the `cdylib` to load (the `foo` in `libfoo.dylib` / `libfoo.so`). It carries a **build** flag whose default is `true` whenever a `Cargo.toml` is present, meaning “build this with cargo before loading”; a plugin that ships a prebuilt library can set it off. This table is how a native plugin that does not follow the “one `cdylib` at the root” convention tells `znative` exactly which library is the plugin.

`[script]` declares a script plugin explicitly. It lists the files to **source** and the directories to prepend to `$fpath`. This is how a script plugin with a non-standard layout — files that are not named `*.plugin.zsh`, or that live in subdirectories detection would not scan — tells `znative` precisely what to load and in what order.

The relationship between the manifest and detection is the thing to hold onto: the manifest is an override, not a requirement. A plugin author who follows the conventions ships nothing. A plugin author whose layout is unusual, or who wants the name/version/description pinned rather than inferred, ships a `znative.toml` and `znative` obeys it exactly. Both paths land in the same store with the same index record; the manifest only changes how `znative` decided what to install, not what installation means.

# Chapter 9 – The `.zshrc` Idiom

Everything in the previous chapters collapses, in practice, to a few lines in a `.zshrc` — one `znative load owner/repo` per plugin, in load order. This chapter is that idiom, because the idiom is the product: the machinery exists so that the startup file can be this plain.

A representative block:

```
znative load zdharma-continuum/history-search-multi-word
znative load zsh-users/zsh-autosuggestions
znative load MenkeTechnologies/zshrs-forgit           # a native Rust plugin
znative load zsh-users/zsh-syntax-highlighting       # kept last, as always
```

Three things about that block are the whole point.

**Load order is source order.** The lines run top to bottom, so plugins load in the order they are written — which matters, because `zsh-syntax-highlighting` must be sourced last (it wraps the line editor and must see the final state of every widget the earlier plugins defined). `znative` does not reorder or defer; the order in the file is the order of the load. This is the same discipline every zsh user already applies to their plugin block, and `znative` preserves it exactly.

**The native line is indistinguishable from the script lines.** The third line loads a compiled Rust plugin through the ABI host, and it is written the same way, in the same place, in the same order as the script plugins around it. There is no separate “native plugin” section, no different verb, no build step you invoke by hand. `znative load` classifies the source, builds it if it is a Rust plugin that needs building, and loads it — and from the `.zshrc`’s point of view it is one more `load` line. The native path is *as routine as the script path*, which is the sentence the whole invention is built to earn.

**First start installs; every start after loads from the store with zero network.** The first time this `.zshrc` runs, each `load` line finds its plugin absent from the store and installs it (Chapter 4’s conditional). Every start after that, each line finds its plugin present and loads it from the store without touching the network. There is no separate `znative install` you run once and a `znative load` you run forever; the one line does both, install-then-load on first sight and load-only thereafter. That is what “no separate install step” means at the level of the file you actually edit.

For the user who would rather install interactively than list every plugin in the startup file, there is the bare form. Run `znative add owner/repo` at a prompt for each plugin you want, and then keep a single unqualified line in `.zshrc`:

```
znative load           # load everything already in the store
```

A bare `znative load` with no argument loads every plugin present in the store, in the store’s order. Either idiom — explicit `load` lines or interactive `add` plus one bare `load` — is the script-plugin workflow zsh users already know, with compiled native plugins folded into it and nothing new to learn.

# Chapter 10 — The Worked Ports

An invention that only works in principle is a proposal. `znative` ships with a set of worked plugin ports that prove the native path end to end — real, useful plugins built against the published ABI, installed by `znative`, running as compiled code. They exist to demonstrate that a stranger *can* build a native plugin and that `znative` *can* install it, using tools people actually want at a prompt.

The ported plugins:

- `forgit` — the interactive-git front-end (fuzzy-searched `git` operations through `fzf`). It ships as the flagship worked zsh → Rust port: the plugin every zsh user recognises, reimplemented against the ABI and loaded as native code rather than sourced as `.zsh`. `MenkeTechnologies/zshrs-forgit` is the self-installing example the front matter opens with.
- `git-fuzzy` — another fuzzy-git workflow tool, ported to the native path.
- `revolver` — the reusable progress-spinner/menu library for zsh scripts.
- `kubectl` — the Kubernetes plugin surface, ported.
- `zsh-z` — the “jump to a frecent directory” plugin (the `z` command), ported to native code.

These are not toy plugins chosen to be easy; they are tools with real per-call work — fuzzy matching, directory frequency scoring, git plumbing — where running as compiled machine code instead of re-interpreted shell text is a measurable difference rather than a cosmetic one. That is the point of the selection: the worked ports are exactly the plugins where the native path *matters*, so that the end-to-end proof is also a demonstration of why anyone would want it.

And critically, they change nothing for the script ecosystem. Script plugins — oh-my-zsh, zinit bundles, any `.zsh` you `source` — keep working exactly as before, installed by the same `znative load` in the same `.zshrc`. The worked ports sit alongside them as evidence of what the native path adds, not as a replacement for the script path anyone has to migrate to. You can run all five native ports and a hundred script plugins from one store, one index, and one block of `load` lines.

# Chapter 11 — The Landscape

`znative` earns “first shell package manager whose unit of installation can be native code” only against the field of managers that came before it, so this chapter is that field and where `znative` sits in it. The claim is narrow and specific on purpose: not “first plugin manager,” not “fastest,” but *first whose unit of installation can be native compiled code rather than shell text*.

The incumbents all share one mechanism:

- **oh-my-zsh** — the framework: a large repository of plugins and themes you enable by name, each a directory of `.zsh` files it `sources`.
- **zinit** — the ergonomics leader: turbo mode, deferred/async loading, its own ice-modifier DSL. Underneath the DSL, it clones repos and sources their files.
- **antigen** — the bundle-declaration manager: `antigen bundle owner/repo`, clone and source.
- **sheldon** — the TOML-configured, lockfile-carrying manager: a declarative plugins file, and again clone-and-source underneath.
- **zplug** — the feature-broad manager (tags, dependencies, parallel install): clone and source.

Every one of them, underneath its configuration model, does the same single act: **clone a git repository and source its `.zsh` files**. They differ — and some differ a great deal — in *ergonomics*: how you declare a plugin, whether loading is deferred, whether there is a lockfile, how parallel the installs are. They do not differ in *kind*. The unit of installation, for all of them, is shell text.

None of them installs a compiled plugin, and the reason is not that they never thought of it — it is that **until the stable ABI of Chapter 2, there was nothing safe to install**. A native shell module before `zshrs` was welded to one shell build; a manager that “installed” one would be shipping a crash waiting for the next shell rebuild. The incumbents install shell text because shell text was the only thing safe to distribute. `znative` installs compiled plugins because `zshrs` first made compiled plugins safe to distribute. The manager did not invent the native path; the ABI did. The manager is what made the native path *installable*, and that is the leg of the claim it owns.

Two omissions distinguish `znative` from some of the incumbents, and both are deliberate. Sheldon carries a lockfile; `znative` does not. Language package managers carry per-project manifests; `znative` does not. `znative` is **global-only** — one store, one index, under `$ZSHRS_HOME/pkg/` — because a shell’s plugins are part of the user’s environment rather than a project’s dependency set, and pretending otherwise would import a reproducibility model that does not fit what shell plugins are. A missing lockfile in a project package manager is a bug; in a shell plugin manager it is a correct read of what the tool is for. `znative` is not “sheldon minus a lockfile” — it is a different category, built on a substrate none of the incumbents had.

# Chapter 12 – Lineage: the Stryke Dependency Manager

`znative` was not designed from nothing. It is **ported from stryke’s own dependency manager** — the package system for strykelang, the parallel Perl-5 superset that shares the fusevm bytecode machine with zshrs — and understanding that lineage explains why `znative`’s native path is as considered as it is.

Stryke’s dependency manager does something more aggressive than any shell plugin manager: it AOT-compile a whole dependency graph to native code. A stryke program that pulls in packages does not carry an interpreter that reads those packages at runtime; the closed-world compiler lowers the whole graph — program plus dependencies — to native registers and a standalone binary. The dependency manager, in that world, is not “fetch some text to interpret later.” It is “fetch source that will be compiled into the artifact.” Native code as the *unit of a dependency* is stryke’s normal case, not an exotic one.

`znative` carries that instinct into the shell. It takes the stryke dependency manager and teaches it to install a compiled plugin the way every other shell manager installs a script — same command surface, same “add this and it is at your prompt” ergonomics, but with a `cdylib` as a legitimate unit of installation rather than only a bag of `.zsh` files. The reason the shell path could accept that idea at all is the reason this book keeps returning to: the published ABI (Chapter 2) gave the shell a safe way to load a foreign compiled artifact, which is the same shape of thing stryke’s manager already dealt in. The lineage is not cosmetic; it is why `znative` treats native code as a first-class package instead of an afterthought bolted onto a script manager.

There is a consistency to it worth naming. zshrs and stryke share fusevm — one bytecode machine, one JIT, one AOT compiler underneath both. It is fitting that they also share a dependency-management instinct: that native compiled code is a normal thing to fetch, build, and install, not a special case. `znative` is that instinct applied to shell plugins, made routine by the ABI that made it safe.

# Chapter 13 — What znative Is Not

A tool is defined by its boundaries as much as its features, and `znative`'s boundaries are chosen, not accidental. This chapter collects the deliberate omissions the book has argued for in pieces, so the shape of the tool is visible whole.

**It is not per-project.** There is no `.znative/` in a repository, no project manifest, no lockfile committed to version control. `znative` keeps one global store and one global index under `$ZSHRS_HOME/pkg/`. A language package manager is right to be per-project — a project's dependency set must be reproducible on someone else's machine — but a shell's plugins are the *user's* environment, not a *project's* contract. Making `znative` global-only is a read of what shell plugins are, not a feature it lacks.

**It is not a lockfile manager.** The store *is* content-addressed — every install carries a `sha256-<hex>` integrity hash in the index — so a record can verify its own content. But there is no separate lockfile pinning a resolved graph across machines, because there is no graph to pin: shell plugins are a flat set the user curated, not a transitive dependency tree a resolver produced. The integrity hash answers “is this the content I installed”; a lockfile would answer “can I reproduce this resolution elsewhere,” and that is a question shell plugins do not ask.

**It is not a new DSL.** There is no ice-modifier syntax, no declarative block with its own grammar, no configuration language to learn. The interface is `znative load owner/repo` in a `.zshrc`, and the optional `znative.toml` a *plugin* ships is TOML, not a bespoke format. The tool's surface is six verbs and one optional manifest.

**It is not chatty.** Errors print `znative: <reason>` on stderr and return non-zero — no friendly explanation, no “did you mean,” no “run `-help`.” It obeys the same doctrine as the rest of `zshrs`: no startup banner, no progress chatter to the terminal, no deprecation nags, no tips. It does what you typed and nothing more.

**It does not replace the script path.** The single most important non-feature. `znative` does not deprecate script plugins, does not ask you to rewrite oh-my-zsh plugins in Rust, does not make the native path mandatory for anything. Script plugins install and load exactly as they always have; the native path is purely additive, for the cases where a plugin wants compiled speed and real type checking instead of re-interpreted text. The invention is not “native instead of script.” It is “native *as well as* script, from one manager, made as routine as the script path always was.”

# Appendix A – Command Reference

| Command                            | Aliases                 | What it does                                                                                                                                                                                                  |
|------------------------------------|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>znative load [SOURCE]</code> | —                       | Load a plugin. If SOURCE is present in the store, load with zero network; if absent, install then load. With <b>no argument</b> , load everything already in the store. The verb a <code>.zshrc</code> calls. |
| <code>znative add SOURCE</code>    | <code>install, i</code> | Resolve, install into the store, record in the index, and load — without needing a <code>.zshrc</code> line. The interactive install.                                                                         |
| <code>znative remove NAME</code>   | —                       | Unload a native plugin ( <code>zmodload -uR</code> ), delete its store copy, and drop its index row.                                                                                                          |
| <code>znative list</code>          | —                       | Print one line per installed plugin: name, version, kind, source. Reads the global index.                                                                                                                     |
| <code>znative info NAME</code>     | —                       | Show the full record for one plugin: store path, integrity hash, and the lib (native) or source files (script).                                                                                               |
| <code>znative update [NAME]</code> | —                       | Re-resolve and reinstall from the recorded source, respecting any pin. One named plugin, or all with no argument.                                                                                             |

Error contract: failures print `znative: <reason>` on stderr and return non-zero.

## Appendix B – Source-Spec Grammar

| Form                                | Classified as     | Network   | Notes                                              |
|-------------------------------------|-------------------|-----------|----------------------------------------------------|
| <code>owner/repo</code>             | GitHub repository | yes       | The common form; bare two-part slug, no scheme.    |
| <code>github:owner/repo</code>      | GitHub repository | yes       | The same, said explicitly.                         |
| <code>git+URL / .../x.git</code>    | arbitrary git URL | yes       | Any git host — GitLab, self-hosted, mirror.        |
| <code>path:DIR</code>               | local directory   | <b>no</b> | Install from disk; the development / offline path. |
| <code>absolute / ./ / ~ path</code> | local directory   | <b>no</b> | Same as <code>path:</code> , recognised by shape.  |

Any remote form may carry an `@ref` suffix to pin a tag, branch, or commit: `owner/repo@v1.2.0`, `github:owner/repo@main`, `git+https://host/x.git@<sha>`. The pin is recorded and honoured by `znative update`.

Clone strategy: shallow by default (`--depth 1 --branch REF`); full-clone fallback only when a shallow branch clone cannot reach a requested commit.

# Appendix C – Store Layout

Root: `$ZSHRS_HOME/pkg/` (default `~/.zshrs/pkg/`). One store per user; global only.

| Path                                             | Role                                                                                                                                               |
|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>store/&lt;name&gt;@&lt;version&gt;/</code> | The installed plugin tree, stripped of <code>.git/</code> and <code>target/</code> .                                                               |
| <code>installed.toml</code>                      | The global index and source of truth: name, version, kind, recorded source, store path, <code>sha256-&lt;hex&gt;</code> integrity hash per plugin. |
| <code>git/</code>                                | Internal scratch — clone working directories.                                                                                                      |
| <code>cache/</code>                              | Internal scratch — fetch caches.                                                                                                                   |
| <code>bin/</code>                                | Internal scratch — helper binaries.                                                                                                                |

There is no per-project manifest and no lockfile. The `sha256-<hex>` hash in the index is the per-record integrity check.

# Appendix D — znative.toml Schema

An optional manifest at a plugin's root. Overrides auto-detection where present.

```
[plugin]
name      = "my-plugin"
version   = "1.0.0"
description = "one-line description"

# exactly one of the two kind tables:

[native]
lib      = "myplugin"    # lib stem: loads libmyplugin.{dylib,so}
build    = true          # default true when a Cargo.toml is present

# - or -

[script]
source    = ["my-plugin.zsh", "extra.zsh"] # files to source, in order
fpath     = ["functions"]                  # dirs to prepend to $fpath
```

- **[plugin]** — identity: **name** and **version** set the **store/<name>@<version>/** key; **description** is the human-readable line.
- **[native]** — a compiled plugin. **lib** names the **cdylib** stem to load; **build** (default **true** with a **Cargo.toml**) says whether to **cargo build --release** before loading.
- **[script]** — a script plugin. **source** lists files to **source**; **fpath** lists directories to prepend to **\$fpath**.

Auto-detection (Chapter 7) covers the common cases; the manifest is the explicit override for unusual layouts or pinned metadata.

# Appendix E – The Doctrine

`znative` obeys the same shell doctrine as the rest of zshrs. The relevant rules for a package manager:

- **No startup banner, no version stripe, no first-run message.** Installing on first shell start is silent to the terminal.
- **No init/cache progress to the terminal.** Clone, build, and store work do not narrate themselves on screen; the shell's prompt is not delayed by chatter.
- **No deprecation nags, no “did you mean,” no tips.** The tool does not editorial on your choices.
- **Errors on stderr, terse.** `znative: <reason>`, non-zero exit. State the error, exit — no friendly explanation, no “run `–help` for more.”
- **First paint equals full functionality.** The native path is not deferred behind a “loading in the background” trick; when a plugin is loaded, it is loaded — the same doctrine that forbids instant-prompt fakery elsewhere in zshrs applies to how plugins come up here.

The manager is small on purpose, silent on purpose, and additive on purpose. It extends the plugin workflow every zsh user already has to compiled native code, and it does so without a second tool, a second mental model, or a single line of chatter the user did not ask for.